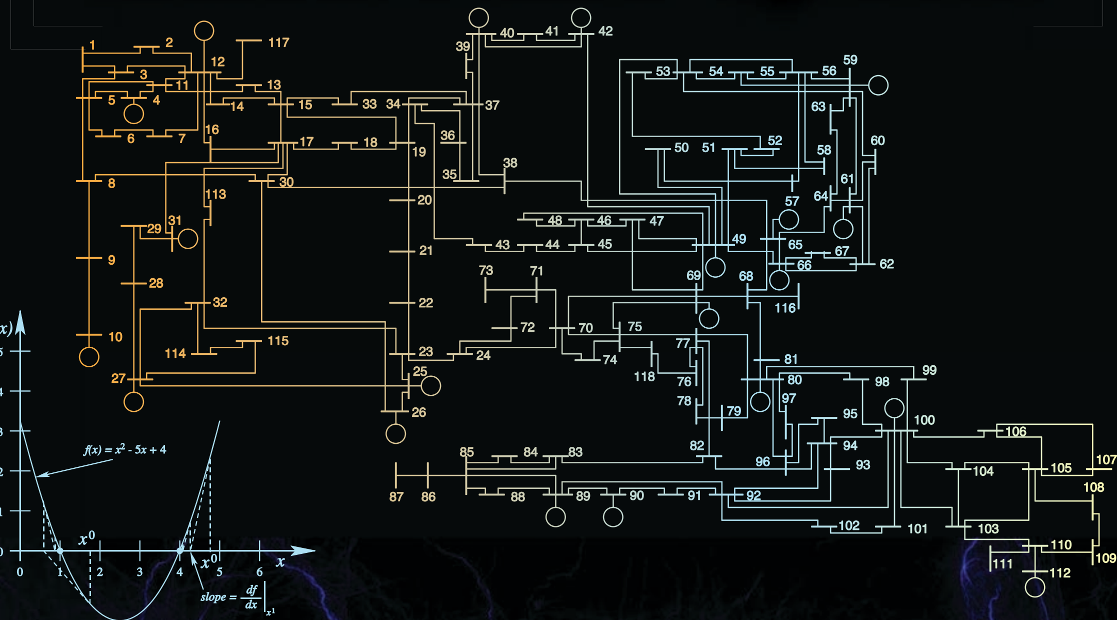
Third Edition

Computational Methods for Electric Power Systems



Mariesa L. Crow

CRC Press
Taylor & Francis Group

Third Edition

Computational Methods for Electric Power Systems

The ELECTRIC POWER ENGINEERING Series
Series Editor Leo L. Grigsby

Published Titles

Computational Methods for Electric Power Systems, Third Edition, Mariesa L. Crow

Electric Power Generation, Transmission, and Distribution, Third Edition, Leonard L. Grigsby

Linear Synchronous Motors: Transportation and Automation Systems, Second Edition, Jacek Gieras, Zbigniew J. Piech, and Bronislaw Tomczuk

The Induction Machines Design Handbook, Second Edition, Ion Boldea and Syed Nasar

Electric Energy Systems: Analysis and Operation, Antonio Gómez-Expósito, Antonio J. Conejo, and Claudio Cañizares

Distribution System Modeling and Analysis, Second Edition, William H. Kersting

Electric Machines, Charles A. Gross

Harmonics and Power Systems, Francisco C. De La Rosa

Electric Drives, Second Edition, Ion Boldea and Syed Nasar

Power System Operations and Electricity Markets, Fred I. Denny and David E. Dismukes

Power Quality, C. Sankaran

Electromechanical Systems, Electric Machines,and Applied Mechatronics, Sergey E. Lyshevski

Electrical Energy Systems, Second Edition, Mohamed E. El-Hawary

Electric Power Substations Engineering, John D. McDonald

Electric Power Transformer Engineering, James H. Harlow

Electric Power Distribution Handbook, Tom Short

Third Edition

Computational Methods for Electric Power Systems

Mariesa L. Crow

Missouri University of Science and Technology, Rolla, USA

CRC Press
Taylor & Francis Group
Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

CRC Press
Taylor & Francis Group
6000 Broken Sound Parkway NW, Suite 300
Boca Raton, FL 33487-2742

Version Date: 20150629

International Standard Book Number-13: 978-1-4987-1160-9 (eBook - PDF)

Visit the Taylor & Francis Web site at
http://www.taylorandfrancis.com

and the CRC Press Web site at
http://www.crcpress.com

To
Jim, David, and Jacob

Contents

Preface to the third edition **xi**

1 Introduction **1**

2 The Solution of Linear Systems **3**
2.1 Gaussian Elimination 4
2.2 LU Factorization 9
2.2.1 LU Factorization with Partial Pivoting 16
2.2.2 LU Factorization with Complete Pivoting 20
2.3 Condition Numbers and Error Propagation 23
2.4 Stationary Iterative Methods 24
2.5 Conjugate Gradient Methods 30
2.6 Generalized Minimal Residual Algorithm 36
2.7 Preconditioners for Iterative Methods 42
2.7.1 Jacobi 42
2.7.2 Symmetric Successive Overrelaxation 43
2.7.3 Symmetric Gauss–Seidel 43
2.7.4 Incomplete LU Factorization 43
2.7.5 Graph Based 44
2.8 Problems 47

3 Systems of Nonlinear Equations **53**
3.1 Fixed-Point Iteration 54
3.2 Newton–Raphson Iteration 61
3.2.1 Convergence Properties 64
3.2.2 The Newton–Raphson for Systems of Nonlinear Equations 65
3.3 Quasi-Newton Methods 68
3.3.1 Secant Method 69
3.3.2 Broyden's Method 72
3.3.3 Modifications to the Newton–Raphson Method 74
3.3.4 Numerical Differentiation 75
3.3.5 Newton–GMRES 79
3.4 Continuation Methods 83
3.5 Power System Applications 86
3.5.1 Power Flow 87

3.5.2 Regulating Transformers 95
3.5.3 Decoupled Power Flow 99
3.5.4 Fast Decoupled Power Flow 101
3.5.5 PV Curves and Continuation Power Flow 105
3.5.6 Three-Phase Power Flow 112
3.6 Problems 113

4 Sparse Matrix Solution Techniques 117
4.1 Storage Methods 118
4.2 Sparse Matrix Representation 127
4.3 Ordering Schemes 127
4.3.1 Scheme 0 141
4.3.2 Scheme I 142
4.3.3 Scheme II 148
4.3.4 Other Schemes 151
4.4 Power System Applications 152
4.5 Problems 156

5 Numerical Integration 163
5.1 One-Step Methods 164
5.1.1 Taylor Series-Based Methods 164
5.1.2 Forward Euler Method 165
5.1.3 Runge–Kutta Methods 165
5.2 Multistep Methods 166
5.2.1 Adams Methods 172
5.2.2 Gear's Methods 175
5.3 Accuracy and Error Analysis 176
5.4 Numerical Stability Analysis 180
5.5 Stiff Systems 187
5.6 Step Size Selection 191
5.7 Differential-Algebraic Equations 198
5.8 Power System Applications 200
5.8.1 Transient Stability Analysis 200
5.8.2 Midterm Stability Analysis 208
5.9 Problems 211

6 Optimization 219
6.1 Least Squares State Estimation 220
6.1.1 Weighted Least Squares Estimation 223
6.1.2 Bad Data Detection 226
6.1.3 Nonlinear Least Squares State Estimation 229
6.2 Linear Programming 230
6.2.1 Simplex Method 231
6.2.2 Interior Point Method 235
6.3 Nonlinear Programming 240

6.3.1 Quadratic Programming 241
6.3.2 Steepest Descent Algorithm 243
6.3.3 Sequential Quadratic Programming Algorithm 248
6.4 Power System Applications 251
6.4.1 Optimal Power Flow 251
6.4.2 State Estimation 262
6.5 Problems 266

7 Eigenvalue Problems 273
7.1 The Power Method 274
7.2 The QR Algorithm 276
7.2.1 Deflation 283
7.2.2 Shifted QR 283
7.2.3 Double Shifted QR 284
7.3 Arnoldi Methods 286
7.4 Singular Value Decomposition 293
7.5 Modal Identification 296
7.5.1 Prony Method 298
7.5.2 The Matrix Pencil Method 301
7.5.3 The Levenberg–Marquardt Method 302
7.5.4 Eigensystem Realization Algorithm 305
7.5.5 Examples 306
7.6 Power System Applications 311
7.6.1 Participation Factors 311
7.7 Problems 312

References 315

Index 321

Preface to the third edition

This book is intended for a graduate level course. The material is first presented in a general algorithmic manner followed by power system applications. Users do not necessarily have to have a power systems background to find this book useful, but many of the comprehensive exercises do require a working knowledge of power system problems and notation.

This new edition has been updated to include new material. Specifically, this new edition has added the following material:

- Updated examples on sparse LU factorization
- Preconditioners for linear iterative methods
- Broyden's method
- Jacobian free Newton–Krylov methods
- Double-shift method for computing complex eigenvalues
- Eigensystem Realization Algorithm

and additional problems and examples.

A course structure would typically include the following chapters in sequence: Chapters 1, 2, and 3. Chapter 2 provides a basic background in linear system solution (both direct and iterative) followed by a discussion of nonlinear system solution in Chapter 3. Chapter 2 can be directly followed by Chapter 4, which covers sparse storage and computation and follows directly from LU factorization. Chapters 5, 6, and 7 can be covered in any order after Chapter 3 depending on the interest of the reader.

Many of the methods presented in this book have commercial software packages that will accomplish their solution far more rigorously with many failsafe attributes included (such as accounting for ill conditioning, etc.). It is not my intent to make students experts in each topic, but rather to develop an appreciation for the methods behind the packages. Many commercial packages provide default settings or choices of parameters for the user; through better understanding of the methods driving the solution, informed users can make better choices and have a better understanding of the situations in which the methods may fail. If this book provides any reader with more confidence in using commercial packages, I have succeeded in my intent.

Mariesa L. Crow
Rolla, Missouri

Lecture notes are available from the CRC Web site:
http://www.crcpress.com/product/isbn/9781498711593

1

Introduction

In today's deregulated environment, the nation's electric power network is being forced to operate in a manner for which it was not intentionally designed. Therefore, system analysis is very important to predict and continually update the operating status of the network. This includes estimating the current power flows and bus voltages (power flow analysis and state estimation), determining the stability limits of the system (continuation power flow, numerical integration for transient stability, and eigenvalue analysis), and minimizing costs (optimal power flow). This book provides an introductory study of the various computational methods that form the basis of many analytical studies in power systems and other engineering and science fields. This book provides the analytical background of the algorithms used in numerous commercial packages. By understanding the theory behind many of the algorithms, the reader/user can better use the software and make more informed decisions (i.e., choice of integration method and step size in simulation packages).

Due to the sheer size of the power grid, hand-based calculations are nearly impossible, and computers offer the only truly viable means for system analysis. The power industry is one of the largest users of computer technology and one of the first industries to embrace the potential of computer analysis when mainframes first became available. Although the first algorithms for power system analysis were developed in the 1940s, it wasn't until the 1960s that computer usage became widespread within the power industry. Many of the analytical techniques and algorithms used today for the simulation and analysis of large systems were originally developed for power system applications.

As power systems increasingly operate under stressed conditions, computer simulation will play a large role in control and security assessment. Commercial packages routinely fail or give erroneous results when used to simulate stressed systems. Understanding of the underlying numerical algorithms is imperative to correctly interpret the results of commercial packages. For example, will the system really exhibit the simulated behavior or is the simulation simply an artifact of a numerical inaccuracy? The educated user can make better judgments about how to compensate for numerical shortcomings in such packages, either by better choice of simulation parameters or by posing the problem in a more numerically tractable manner. This book will provide the background for a number of widely used numerical algorithms that underlie many commercial packages for power system analysis and design.

This book is intended to be used as a text in conjunction with a semester-long graduate level course in computational algorithms. While the majority of examples in this text are based on power system applications, the theory is presented in a general manner so as to be applicable to a wide range of engineering systems. Although some knowledge of power system engineering may be required to fully appreciate the subtleties of some of the illustrations, such knowledge is not a prerequisite for understanding the algorithms themselves. The text and examples are used to provide an introduction to a wide range of numerical methods without being an exhaustive reference. Many of the algorithms presented in this book have been the subject of numerous modifications and are still the object of on-going research. As this text is intended to provide a foundation, many of these new advances are not explicitly covered, but are rather given as references for the interested reader. The examples in this text are intended to be simple and thorough enough to be reproduced easily. Most "real world" problems are much larger in size and scope, but the methodologies presented in this text should sufficiently prepare the reader to cope with any difficulties he/she may encounter.

Most of the examples in this text were produced using code written in MATLAB®. Although this was the platform used by the author, in practice, any computer language may be used for implementation. There is no practical reason for a preference for any particular platform or language.

2

The Solution of Linear Systems

In many branches of engineering and science it is desirable to be able to mathematically determine the state of a system based on a set of physical relationships. These physical relationships may be determined from characteristics such as circuit topology, mass, weight, or force, to name a few. For example, the injected currents, network topology, and branch impedances govern the voltages at each node of a circuit. In many cases, the relationship between the known, or input, quantities and the unknown, or output, states is a linear relationship. Therefore, a linear system may be generically modeled as

$$Ax = b \tag{2.1}$$

where b is the $n \times 1$ vector of known quantities, x is the $n \times 1$ unknown state vector, and A is the $n \times n$ matrix that relates x to b. For the time being, it will be assumed that the matrix A is invertible, or non-singular; thus each vector b will yield a unique corresponding vector x. Thus the matrix A^{-1} exists and

$$x^* = A^{-1}b \tag{2.2}$$

is the unique solution to Equation (2.1).

The natural approach to solving Equation (2.1) is to directly calculate the inverse of A and multiply it by the vector b. One method to calculate A^{-1} is to use *Cramer's* rule:

$$A^{-1}(i,j) = \frac{1}{det(A)}\left(A_{ij}\right)^T \quad \text{for } i = 1,\ldots,n, j = 1,\ldots,n \tag{2.3}$$

where $A^{-1}(i,j)$ is the ijth entry of A^{-1} and A_{ij} is the cofactor of each entry a_{ij} of A. This method requires the calculation of $(n+1)$ determinants, which results in $2(n+1)!$ multiplications to find A^{-1}! For large values of n, the calculation requirement grows too rapidly for computational tractability; thus alternative approaches have been developed.

Basically, there are two approaches to solving Equation (2.1):

- *Direct methods*, or elimination methods, find the exact solution (within the accuracy of the computer) through a finite number of arithmetic operations. The solution x of a direct method would be completely accurate were it not for computer roundoff errors.

- *Iterative methods*, on the other hand, generate a sequence of (hopefully) progressively improving approximations to the solution based on the application of the same computational procedure at each step. The iteration is terminated when an approximate solution is obtained having some prespecified accuracy or when it is determined that the iterates are not improving.

The choice of solution methodology usually relies on the structure of the system under consideration. Certain systems lend themselves more amenably to one type of solution method versus the other. In general, direct methods are best for full matrices, whereas iterative methods are better for matrices that are large and sparse. But, as with most generalizations, there are notable exceptions to this rule of thumb.

2.1 Gaussian Elimination

An alternate method for solving Equation (2.1) is to solve for x without calculating A^{-1} explicitly. This approach is a *direct method* of linear system solution, since x is found directly. One common direct method is the method of *Gaussian elimination.* The basic idea behind Gaussian elimination is to use the first equation to eliminate the first unknown from the remaining equations. This process is repeated sequentially for the second unknown, the third unknown, etc., until the elimination process is completed. The nth unknown is then calculated directly from the input vector b. The unknowns are then recursively substituted back into the equations until all unknowns have been calculated.

Gaussian elimination is the process by which the augmented $n \times (n+1)$ matrix

$$[A \mid b]$$

is converted to the $n \times (n+1)$ matrix

$$[I \mid b^*]$$

through a series of elementary row operations, where

$$\begin{aligned}
Ax &= b \\
A^{-1}Ax &= A^{-1}b \\
Ix &= A^{-1}b = b^* \\
x^* &= b^*
\end{aligned}$$

Thus, if a series of elementary row operations exist that can transform the matrix A into the identity matrix I, then the application of the same set of elementary row operations will also transform the vector b into the solution vector x^*.

An elementary row operation consists of one of three possible actions that can be applied to a matrix:

- interchange any two rows of the matrix
- multiply any row by a constant
- take a linear combination of rows and add it to another row

The elementary row operations are chosen to transform the matrix A into an upper triangular matrix that has ones on the diagonal and zeros in the subdiagonal positions. This process is known as the *forward elimination* step. Each step in the forward elimination can be obtained by successively premultiplying the matrix A by an elementary matrix ξ, where ξ is the matrix obtained by performing an elementary row operation on the identity matrix.

Example 2.1
Find a sequence of elementary matrices that, when applied to the following matrix, will produce an upper triangular matrix.

$$A = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 2 & 1 & 2 & 3 \\ 4 & 3 & 5 & 8 \\ 9 & 2 & 7 & 4 \end{bmatrix}$$

Solution 2.1 To upper triangularize the matrix, the elementary row operations will need to systematically zero out each column below the diagonal. This can be achieved by replacing each row of the matrix below the diagonal with the difference of the row itself and a constant times the diagonal row, where the constant is chosen to result in a zero sum in the column under the diagonal. Therefore, row 2 of A is replaced by (row 2 minus 2(row 1)) and the elementary matrix is

$$\xi_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

and

$$\xi_1 A = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 0 & -5 & -6 & -13 \\ 4 & 3 & 5 & 8 \\ 9 & 2 & 7 & 4 \end{bmatrix}$$

Note that all rows except row 2 remain the same and row 2 now has a 0 in the column under the first diagonal. Similarly, the two elementary matrices that complete the elimination of the first column are

$$\xi_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -4 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\xi_3 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -9 & 0 & 0 & 1 \end{bmatrix}$$

and

$$\xi_3\xi_2\xi_1 A = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 0 & -5 & -6 & -13 \\ 0 & -9 & -11 & -24 \\ 0 & -25 & -29 & -68 \end{bmatrix} \tag{2.4}$$

The process is now applied to the second column to zero out everything below the second diagonal and scale the diagonal to one. Therefore,

$$\xi_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -\frac{9}{5} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\xi_5 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -\frac{25}{5} & 0 & 1 \end{bmatrix}$$

$$\xi_6 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -\frac{1}{5} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Similarly,

$$\xi_6\xi_5\xi_4\xi_3\xi_2\xi_1 A = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 0 & 1 & \frac{6}{5} & \frac{13}{5} \\ 0 & 0 & -\frac{1}{5} & -\frac{3}{5} \\ 0 & 0 & 1 & -3 \end{bmatrix} \tag{2.5}$$

Similarly,

$$\xi_7 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 5 & 1 \end{bmatrix}$$

$$\xi_8 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -5 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

yielding

$$\xi_8\xi_7\xi_6\xi_5\xi_4\xi_3\xi_2\xi_1 A = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 0 & 1 & \frac{6}{5} & \frac{13}{5} \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & -6 \end{bmatrix} \tag{2.6}$$

Finally

$$\xi_9 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -\frac{1}{6} \end{bmatrix}$$

and

$$\xi_9\xi_8\xi_7\xi_6\xi_5\xi_4\xi_3\xi_2\xi_1 A = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 0 & 1 & \frac{6}{5} & \frac{13}{5} \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.7}$$

which completes the upper triangularization process. ■

Once an upper triangular matrix has been achieved, the solution vector x^* can be found by successive substitution (or *back substitution*) of the states.

Example 2.2

Using the upper triangular matrix of Example 2.1, find the solution to

$$\begin{bmatrix} 1 & 3 & 4 & 8 \\ 2 & 1 & 2 & 3 \\ 4 & 3 & 5 & 8 \\ 9 & 2 & 7 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Solution 2.2 Note that the product of a series of lower triangular matrices is lower triangular; therefore, the product

$$W = \xi_9\xi_8\xi_7\xi_6\xi_5\xi_4\xi_3\xi_2\xi_1 \tag{2.8}$$

is lower triangular. Since the application of the elementary matrices to the matrix A results in an upper triangular matrix, then

$$WA = U \tag{2.9}$$

where U is the upper triangular matrix that results from the forward elimination process. Premultiplying Equation (2.1) by W yields

$$WAx = Wb \tag{2.10}$$
$$Ux = Wb \tag{2.11}$$
$$= b' \tag{2.12}$$

where $Wb = b'$.

From Example 2.1:

$$W = \begin{bmatrix} 1 & 0 & 0 & 0 \\ \frac{2}{5} & -\frac{1}{5} & 0 & 0 \\ 2 & 9 & -5 & 0 \\ \frac{1}{6} & \frac{14}{6} & -\frac{5}{6} & -\frac{1}{6} \end{bmatrix}$$

and

$$b' = W \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ \frac{1}{5} \\ 6 \\ \frac{3}{2} \end{bmatrix}$$

Thus

$$\begin{bmatrix} 1 & 3 & 4 & 8 \\ 0 & 1 & \frac{6}{5} & \frac{13}{5} \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ \frac{1}{5} \\ 6 \\ \frac{3}{2} \end{bmatrix} \tag{2.13}$$

By inspection, $x_4 = \frac{3}{2}$. The third row yields

$$x_3 = 6 - 3x_4 \tag{2.14}$$

Substituting the value of x_4 into Equation (2.14) yields $x_3 = \frac{3}{2}$. Similarly,

$$x_2 = \frac{1}{5} - \frac{6}{5}x_3 - \frac{13}{5}x_4 \tag{2.15}$$

and substituting x_3 and x_4 into Equation (2.15) yields $x_2 = -\frac{11}{2}$. Solving for x_1 in a similar manner produces

$$x_1 = 1 - 3x_2 - 4x_3 - 8x_4 \tag{2.16}$$
$$= -\frac{1}{2} \tag{2.17}$$

Thus

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} -1 \\ -11 \\ 3 \\ 3 \end{bmatrix} \quad \blacksquare$$

The solution methodology of successively substituting values of x back into the equation as they are found gives rise to the name of *back substitution* for this step of Gaussian elimination. Therefore, Gaussian elimination consists of two main steps: forward elimination and back substitution. Forward elimination is the process of transforming the matrix A into triangular factors. Back substitution is the process by which the unknown vector x is found from the input vector b and the factors of A. Gaussian elimination also provides the framework under which the LU factorization process is developed.

2.2 LU Factorization

The forward elimination step of Gaussian elimination produces a series of upper and lower triangular matrices that are related to the A matrix as given in Equation (2.9). The matrix W is a lower triangular matrix and U is an upper triangular matrix with ones on the diagonal. Recall that the inverse of a lower triangular matrix is also a lower triangular matrix; therefore, if

$$L \triangleq W^{-1}$$

then

$$A = LU$$

The matrices L and U give rise to the name of the factorization/elimination algorithm known as "LU factorization." In fact, given any nonsingular matrix A, there exists some permutation matrix P (possibly $P = I$) such that

$$LU = PA \tag{2.18}$$

where U is upper triangular with unit diagonals, L is lower triangular with nonzero diagonals, and P is a matrix of ones and zeros obtained by rearranging the rows and columns of the identity matrix. Once a proper matrix P is chosen, this factorization is unique [8]. Once P, L, and U are determined, then the system

$$Ax = b \tag{2.19}$$

can be solved expeditiously. Premultiplying Equation (2.19) by the matrix P yields

$$PAx = Pb = b' \tag{2.20}$$

$$LUx = b' \tag{2.21}$$

where b' is just a rearrangement of the vector b. Introducing a "dummy" vector y such that

$$Ux = y \tag{2.22}$$

thus

$$Ly = b' \tag{2.23}$$

Consider the structure of Equation (2.23):

$$\begin{bmatrix} l_{11} & 0 & 0 & \cdots & 0 \\ l_{21} & l_{22} & 0 & \cdots & 0 \\ l_{31} & l_{32} & l_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \cdots & l_{nn} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} b'_1 \\ b'_2 \\ b'_3 \\ \vdots \\ b'_n \end{bmatrix}$$

The elements of the vector y can be found by straightforward substitution:

$$\begin{aligned} y_1 &= \frac{b'_1}{l_{11}} \\ y_2 &= \frac{1}{l_{22}} \left(b'_2 - l_{21} y_1 \right) \\ y_3 &= \frac{1}{l_{33}} \left(b'_3 - l_{31} y_1 - l_{32} y_2 \right) \\ &\vdots \\ y_n &= \frac{1}{l_{nn}} \left(b'_n - \sum_{j=1}^{n-1} l_{nj} y_j \right) \end{aligned}$$

After the vector y has been found, then x can be easily found from

$$\begin{bmatrix} 1 & u_{12} & u_{13} & \cdots & u_{1n} \\ 0 & 1 & u_{23} & \cdots & u_{2n} \\ 0 & 0 & 1 & \cdots & u_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \vdots & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix}$$

Similarly, the solution vector x can be found by backward substitution:

$$\begin{aligned} x_n &= y_n \\ x_{n-1} &= y_{n-1} - u_{n-1,n} x_n \\ x_{n-2} &= y_{n-2} - u_{n-2,n} x_n - u_{n-2,n-1} x_{n-1} \\ &\vdots \\ x_1 &= y_1 - \sum_{j=2}^{n} u_{1j} x_j \end{aligned}$$

The value of LU factorization is that, once A is factored into the upper and lower triangular matrices, the solution for the solution vector x is straightforward. Note that the inverse to A is never explicitly found.

Several methods for computing the LU factors exist and each method has its advantages and disadvantages. One common factorization approach is

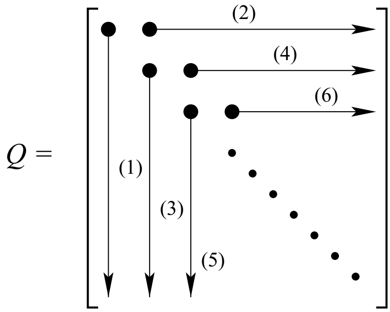


FIGURE 2.1
Order of calculating columns and rows of Q

known as *Crout's* algorithm for finding the LU factors [8]. Let the matrix Q be defined as

$$Q \triangleq L + U - I = \begin{bmatrix} l_{11} & u_{12} & u_{13} & \cdots & u_{1n} \\ l_{21} & l_{22} & u_{23} & \cdots & u_{2n} \\ l_{31} & l_{32} & l_{33} & \cdots & u_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \cdots & l_{nn} \end{bmatrix} \tag{2.24}$$

Crout's algorithm computes the elements of Q first by column and then row, as shown in Figure 2.1. Each element q_{ij} of Q depends only on the a_{ij} entry of A and previously computed values of Q.

Crout's Algorithm for Computing LU from A

1. Initialize Q to the zero matrix. Let $j = 1$.

2. Complete the jth column of Q (jth column of L) as

$$q_{kj} = a_{kj} - \sum_{i=1}^{j-1} q_{ki} q_{ij} \quad \text{for } k = j, \ldots, n \tag{2.25}$$

3. If $j = n$, then stop.

4. Assuming that $q_{jj} \neq 0$, complete the jth row of Q (jth row of U) as

$$q_{jk} = \frac{1}{q_{jj}} \left(a_{jk} - \sum_{i=1}^{j-1} q_{ji} q_{ik} \right) \quad \text{for } k = j+1, \ldots, n \tag{2.26}$$

5. Set $j = j + 1$. Go to step 2.

Once the LU factors are found, then the dummy vector y can be found by forward substitution.

Forward Substitution

$$y_k = \frac{1}{q_{kk}}\left(b_k - \sum_{j=1}^{k-1} q_{kj} y_j\right) \quad \text{for } k = 1, \ldots, n \tag{2.27}$$

Similarly, the solution vector x can be found by backward substitution.

Backward Substitution:

$$x_k = y_k - \sum_{j=k+1}^{n} q_{kj} x_j \quad \text{for } k = n, n-1, \ldots, 1 \tag{2.28}$$

One measure of the computation involved in the LU factorization process is to count the number of multiplications and divisions required to find the solution, since these are both floating point operations. Computing the jth column of Q (jth column of L) requires

$$\sum_{j=1}^{n}\sum_{k=j}^{n}(j-1)$$

multiplications and divisions. Similarly, computing the jth row of Q (jth row of U) requires

$$\sum_{j=1}^{n-1}\sum_{k=j+1}^{n} j$$

multiplications and divisions. The forward substitution step requires

$$\sum_{j=1}^{n} j$$

and the backward substitution step requires

$$\sum_{j=1}^{n}(n-j)$$

multiplications and divisions. Taken together, the LU factorization procedure requires

$$\frac{1}{3}\left(n^3 - n\right)$$

and the substitution steps require n^2 multiplications and divisions. Therefore, the whole process of solving the linear system of Equation (2.1) requires a total of

$$\frac{1}{3}\left(n^3 - n\right) + n^2 \tag{2.29}$$

multiplications and divisions. Compare this to the requirements of Cramer's rule, which requires $2(n+1)!$ multiplications and divisions. Obviously, for a system of any significant size, it is far more computationally efficient to use LU factorization and forward/backward substitution to find the solution x.

Example 2.3

Using LU factorization with forward and backward substitution, find the solution to the system of Example 2.2.

Solution 2.3 The first step is to find the LU factors of the A matrix:

$$A = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 2 & 1 & 2 & 3 \\ 4 & 3 & 5 & 8 \\ 9 & 2 & 7 & 4 \end{bmatrix}$$

Starting with $j = 1$, Equation (2.25) indicates that the elements of the first column of Q are identical to the elements of the first column of A. Similarly, according to Equation (2.26), the first row of Q becomes

$$\begin{aligned} q_{12} &= \frac{a_{12}}{q_{11}} = \frac{3}{1} = 3 \\ q_{13} &= \frac{a_{13}}{q_{11}} = \frac{4}{1} = 4 \\ q_{14} &= \frac{a_{14}}{q_{11}} = \frac{8}{1} = 8 \end{aligned}$$

Thus, for $j = 1$, the Q matrix becomes

$$Q = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 2 & & & \\ 4 & & & \\ 9 & & & \end{bmatrix}$$

For $j = 2$, the second column and row of Q below and to the right of the diagonal, respectively, will be calculated. For the second column of Q:

$$\begin{aligned} q_{22} &= a_{22} - q_{21}q_{12} = 1 - (2)(3) = -5 \\ q_{32} &= a_{32} - q_{31}q_{12} = 3 - (4)(3) = -9 \\ q_{42} &= a_{42} - q_{41}q_{12} = 2 - (9)(3) = -25 \end{aligned}$$

Each element of Q uses the corresponding element of A and elements of Q that have been previously computed. Note also that the inner indices of the products are always the same and the outer indices are the same as the indices of the element being computed. This holds true for both column and row calculations. The second row of Q is computed

$$q_{23} = \frac{1}{q_{22}}\left(a_{23} - q_{21}q_{13}\right) = \frac{1}{-5}\left(2 - (2)(4)\right) = \frac{6}{5}$$
$$q_{24} = \frac{1}{q_{22}}\left(a_{24} - q_{21}q_{14}\right) = \frac{1}{-5}\left(3 - (2)(8)\right) = \frac{13}{5}$$

After $j = 2$, the Q matrix becomes

$$Q = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 2 & -5 & \frac{6}{5} & \frac{13}{5} \\ 4 & -9 & & \\ 9 & -25 & & \end{bmatrix}$$

Continuing on for $j = 3$, the third column of Q is calculated

$$q_{33} = a_{33} - (q_{31}q_{13} + q_{32}q_{23}) = 5 - \left((4)(4) + (-9)\frac{6}{5}\right) = -\frac{1}{5}$$
$$q_{43} = a_{43} - (q_{41}q_{13} + q_{42}q_{23}) = 7 - \left((9)(4) + (-25)\frac{6}{5}\right) = 1$$

and the third row of Q becomes

$$\begin{aligned} q_{34} &= \frac{1}{q_{33}}\left(a_{34} - (q_{31}q_{14} + q_{32}q_{24})\right) \\ &= (-5)\left(8 - \left((4)(8) + (-9)\left(\frac{13}{5}\right)\right)\right) = 3 \end{aligned}$$

yielding

$$Q = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 2 & -5 & \frac{6}{5} & \frac{13}{5} \\ 4 & -9 & -\frac{1}{5} & 3 \\ 9 & -25 & 1 & \end{bmatrix}$$

Finally, for $j = 4$, the final diagonal element is found:

$$\begin{aligned} q_{44} &= a_{44} - (q_{41}q_{14} + q_{42}q_{24} + q_{43}q_{34}) \\ &= 4 - \left((9)(8) + (-25)\left(\frac{13}{5}\right) + (3)(1)\right) = -6 \end{aligned}$$

Thus

$$Q = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 2 & -5 & \frac{6}{5} & \frac{13}{5} \\ 4 & -9 & -\frac{1}{5} & 3 \\ 9 & -25 & 1 & -6 \end{bmatrix}$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & -5 & 0 & 0 \\ 4 & -9 & -\frac{1}{5} & 0 \\ 9 & -25 & 1 & -6 \end{bmatrix}$$

$$U = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 0 & 1 & \frac{6}{5} & \frac{13}{5} \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

One method of checking the correctness of the solution is to check if $LU = A$, which in this case it does.

Once the LU factors have been found, then the next step in the solution process is forward elimination using the L matrix and the b vector to find the dummy vector y. Using forward substitution to solve $Ly = b$ for y:

$$\begin{aligned}
y_1 &= \frac{b_1}{L_{11}} = \frac{1}{1} = 1 \\
y_2 &= \frac{(b_2 - L_{21}y_1)}{L_{22}} = \frac{(1-(2)(1))}{-5} = \frac{1}{5} \\
y_3 &= \frac{(b_3 - (L_{31}y_1 + L_{32}y_2))}{L_{33}} = (-5)\left(1 - \left((4)(1) + (-9)\frac{1}{5}\right)\right) = 6 \\
y_4 &= \frac{(b_4 - (L_{41}y_1 + L_{42}y_2 + L_{43}y_3))}{L_{44}} \\
&= \frac{\left(1 - \left((9)(1) + (-25)\left(\frac{1}{5}\right) + (1)(6)\right)\right)}{-6} = \frac{3}{2}
\end{aligned}$$

Thus

$$y = \begin{bmatrix} 1 \\ \frac{1}{5} \\ 6 \\ \frac{3}{2} \end{bmatrix}$$

Similarly, backward substitution is then applied to $Ux = y$ to find the solution vector x:

$$\begin{aligned}
x_4 &= y_4 = \frac{3}{2} \\
x_3 &= y_3 - U_{34}x_4 = 6 - (3)\left(\frac{3}{2}\right) = \frac{3}{2} \\
x_2 &= y_2 - (U_{24}x_4 + U_{23}x_3) = \frac{1}{5} - \left(\left(\frac{13}{5}\right)\left(\frac{3}{2}\right) + \left(\frac{6}{5}\right)\left(\frac{3}{2}\right)\right) = -\frac{11}{2} \\
x_1 &= y_1 - (U_{14}x_4 + U_{13}x_3 + U_{12}x_2) \\
&= 1 - \left((8)\left(\frac{3}{2}\right) + (4)\left(\frac{3}{2}\right) + (3)\left(-\frac{11}{2}\right)\right) = -\frac{1}{2}
\end{aligned}$$

yielding the final solution vector

$$x = \frac{1}{2}\begin{bmatrix} -1 \\ -11 \\ 3 \\ 3 \end{bmatrix}$$

which is the same solution found by Gaussian elimination and backward substitution in Example 2.2. A quick check to verify the correctness of the solution is to substitute the solution vector x back into the linear system $Ax = b$. ■

2.2.1 LU Factorization with Partial Pivoting

The LU factorization process presented assumes that the diagonal element is nonzero. Not only must the diagonal element be nonzero, it must be of the same order of magnitude as the other nonzero elements. Consider the solution of the following linear system

$$\begin{bmatrix} 10^{-10} & 1 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \end{bmatrix} \tag{2.30}$$

By inspection, the solution to this linear system is

$$x_1 \approx 2$$
$$x_2 \approx 1$$

The LU factors for A are

$$L = \begin{bmatrix} 10^{-10} & 0 \\ 2 & \left(1 - 2 \times 10^{10}\right) \end{bmatrix}$$
$$U = \begin{bmatrix} 1 & 10^{10} \\ 0 & 1 \end{bmatrix}$$

Applying forward elimination to solve for the dummy vector y yields

$$y_1 = 10^{10}$$
$$y_2 = \frac{\left(5 - 2 \times 10^{10}\right)}{\left(1 - 2 \times 10^{10}\right)} \approx 1$$

Back substituting y into $Ux = y$ yields

$$x_2 = y_2 \approx 1$$
$$x_1 = 10^{10} - 10^{10}x_2 \approx 0$$

The solution for x_2 is correct, but the solution for x_1 is considerably off. Why did this happen? The problem with the equations arranged the way they are in Equation (2.30) is that 10^{-10} is too near zero for most computers. However, if the equations are rearranged such that

$$\begin{bmatrix} 2 & 1 \\ 10^{-10} & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \end{bmatrix} \tag{2.31}$$

then the LU factors become

$$\begin{aligned} L &= \begin{bmatrix} 2 & 0 \\ 10^{-10} & \left(1 - \frac{1}{2} \times 10^{-10}\right) \end{bmatrix} \\ U &= \begin{bmatrix} 1 & \frac{1}{2} \\ 0 & 1 \end{bmatrix} \end{aligned}$$

The dummy vector y becomes

$$\begin{aligned} y_1 &= \frac{5}{2} \\ y_2 &= \frac{\left(1 - \frac{5}{2} \times 10^{-10}\right)}{\left(1 - \frac{1}{2} \times 10^{-10}\right)} \approx 1 \end{aligned}$$

and by back substitution, x becomes

$$\begin{aligned} x_2 &\approx 1 \\ x_1 &\approx \frac{5}{2} - \frac{1}{2}(1) = 2 \end{aligned}$$

which is the solution obtained by inspection of the equations. Therefore, even though the diagonal entry may not be exactly zero, it is still good practice to rearrange the equations such that the largest magnitude element lies on the diagonal. This process is known as *pivoting* and gives rise to the permutation matrix P of Equation (2.18).

Since Crout's algorithm computes the Q matrix by column and row with increasing index, only *partial pivoting* can be used, that is, only the rows of Q (and correspondingly A) can be exchanged. The columns must remain static. To choose the best pivot, the column beneath the jth diagonal (at the jth step in the LU factorization) is searched for the element with the largest absolute value. The corresponding row and the jth row are then exchanged. The pivoting strategy may be succinctly expressed as

Partial Pivoting Strategy

1. At the jth step of LU factorization, choose the kth row as the exchange row such that
$$|q_{jj}| = max\, |q_{kj}| \text{ for } k = j, \ldots, n \tag{2.32}$$
2. Exchange rows and update A, P, and Q correspondingly.

The permutation matrix P is composed of ones and zeros and is obtained as the product of a series of elementary permutation matrices $P^{j,k}$ which represent the exchange of rows j and k. The elementary permutation matrix $P^{j,k}$, shown in Figure 2.2, is obtained from the identify matrix by interchanging rows j and k. A pivot is achieved by the premultiplication of a properly chosen $P^{j,k}$. Since this is only an interchange of rows, the order of the unknown vector does not change.

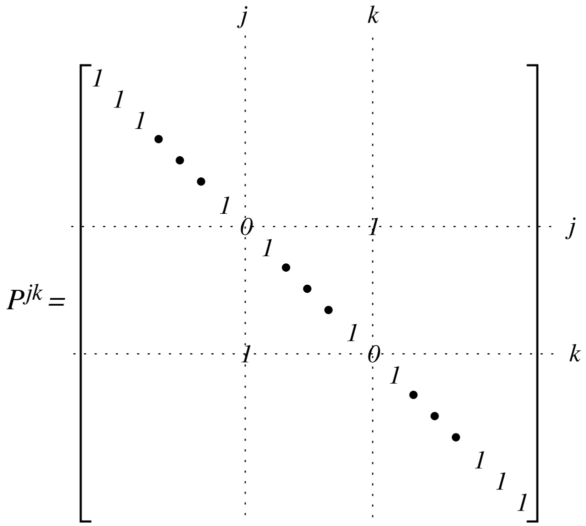


FIGURE 2.2
Elementary permutation matrix $P^{j,k}$

Example 2.4
Repeat Example 2.3 using partial pivoting.

Solution 2.4 The A matrix is repeated here for convenience.

$$A = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 2 & 1 & 2 & 3 \\ 4 & 3 & 5 & 8 \\ 9 & 2 & 7 & 4 \end{bmatrix}$$

For $j = 1$, the first column of Q is exactly the first column of A. Applying the pivoting strategy of Equation (2.32), the q_{41} element has the largest magnitude of the first column; therefore, rows four and one are exchanged. The elementary permutation matrix $P^{1,4}$ is

$$P^{1,4} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

The corresponding A matrix becomes

$$A = \begin{bmatrix} 9 & 2 & 7 & 4 \\ 2 & 1 & 2 & 3 \\ 4 & 3 & 5 & 8 \\ 1 & 3 & 4 & 8 \end{bmatrix}$$

and Q at the $j = 1$ step

$$Q = \begin{bmatrix} 9 & \frac{2}{9} & \frac{7}{9} & \frac{4}{9} \\ 2 & & & \\ 4 & & & \\ 1 & & & \end{bmatrix}$$

At $j = 2$, the calculation of the second column of Q yields

$$Q = \begin{bmatrix} 9 & \frac{2}{9} & \frac{7}{9} & \frac{4}{9} \\ 2 & \frac{5}{9} & & \\ 4 & \frac{19}{9} & & \\ 1 & \frac{25}{9} & & \end{bmatrix}$$

Searching the elements in the jth column below the diagonal, the fourth row of the jth (i.e., second) column once again yields the largest magnitude. Therefore, rows two and four must be exchanged, yielding the elementary permutation matrix $P^{2,4}$

$$P^{2,4} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Similarly, the updated A is

$$\begin{bmatrix} 9 & 2 & 7 & 4 \\ 1 & 3 & 4 & 8 \\ 4 & 3 & 5 & 8 \\ 2 & 1 & 2 & 3 \end{bmatrix}$$

which yields the following Q

$$Q = \begin{bmatrix} 9 & \frac{2}{9} & \frac{7}{9} & \frac{4}{9} \\ 1 & \frac{25}{9} & \frac{29}{25} & \frac{68}{25} \\ 4 & \frac{19}{9} & & \\ 2 & \frac{5}{9} & & \end{bmatrix}$$

For $j = 3$, the calculation of the third column of Q yields

$$Q = \begin{bmatrix} 9 & \frac{2}{9} & \frac{7}{9} & \frac{4}{9} \\ 1 & \frac{25}{9} & \frac{29}{25} & \frac{68}{25} \\ 4 & \frac{19}{9} & -\frac{14}{25} & \\ 2 & \frac{5}{9} & -\frac{1}{5} & \end{bmatrix}$$

In this case, the diagonal element has the largest magnitude, so no pivoting is required. Continuing with the calculation of the 3rd row of Q yields

$$Q = \begin{bmatrix} 9 & \frac{2}{9} & \frac{7}{9} & \frac{4}{9} \\ 1 & \frac{25}{9} & \frac{29}{25} & \frac{68}{25} \\ 4 & \frac{19}{9} & -\frac{14}{25} & -\frac{12}{14} \\ 2 & \frac{5}{9} & -\frac{1}{5} & \end{bmatrix}$$

Finally, calculating q_{44} yields the final Q matrix

$$Q = \begin{bmatrix} 9 & \frac{2}{9} & \frac{7}{9} & \frac{4}{9} \\ 1 & \frac{25}{9} & \frac{29}{25} & \frac{68}{25} \\ 4 & \frac{19}{9} & -\frac{14}{25} & -\frac{12}{14} \\ 2 & \frac{5}{9} & -\frac{1}{5} & \frac{3}{7} \end{bmatrix}$$

The permutation matrix P is found by multiplying together the two elementary permutation matrices:

$$\begin{aligned} P &= P^{2,4} P^{1,4} I \\ &= \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \end{aligned}$$

The results can be checked to verify that $PA = LU$. The forward and backward substitution steps are carried out on the modified vector $b' = Pb$. ■

2.2.2 LU Factorization with Complete Pivoting

An alternate LU factorization that allows complete pivoting is *Gauss'* method. In this approach, two permutation matrices are developed: one for row exchange as in partial pivoting, and a second matrix for column exchange. In this approach, the LU factors are found such that

$$P_1 A P_2 = LU \tag{2.33}$$

Therefore, to solve the linear system of equations $Ax = b$ requires that a slightly different approach be used. As with partial pivoting, the permutation matrix P_1 premultiplies the linear system:

$$P_1 A x = P_1 b = b' \tag{2.34}$$

Now, define a new vector z such that

$$x = P_2 z \tag{2.35}$$

Then substituting Equation (2.35) into Equation (2.34) yields

$$\begin{aligned} P_1 A P_2 z &= P_1 b = b' \\ LUz &= b' \end{aligned} \tag{2.36}$$

where Equation (2.36) can be solved using forward and backward substitution for z. Once z is obtained, then the solution vector x follows from Equation (2.35).

In complete pivoting, both rows and columns may be interchanged to place the largest element (in magnitude) on the diagonal at each step in the LU factorization process. The pivot element is chosen from the remaining elements below and to the right of the diagonal.

Complete Pivoting Strategy

1. At the jth step of LU factorization, choose the pivot element such that

$$|q_{jj}| = max\,|q_{kl}| \text{ for } k = j, \ldots, n, \text{and } l = j, \ldots, n \tag{2.37}$$

2. Exchange rows and update A, P, and Q correspondingly.

Gauss' Algorithm for Computing LU from A

1. Initialize Q to the zero matrix. Let $j = 1$.

2. Set the jth column of Q (jth column of L) to the jth column of the reduced matrix $A^{(j)}$, where $A^{(1)} = A$, and

$$q_{kj} = a_{kj}^{(j)} \text{ for } k = j, \ldots, n \tag{2.38}$$

3. If $j = n$, then stop.

4. Assuming that $q_{jj} \neq 0$, set the jth row of Q (jth row of U) as

$$q_{jk} = \frac{a_{jk}^{(j)}}{q_{jj}} \text{ for } k = j + 1, \ldots, n \tag{2.39}$$

5. Update $A^{(j+1)}$ from $A^{(j)}$ as

$$a_{ik}^{(j+1)} = a_{ik}^{(j)} - q_{ij} q_{jk} \text{ for } i = j + 1, \ldots, n, \text{and } k = j + 1, \ldots, n \tag{2.40}$$

6. Set $j = j + 1$. Go to step 2.